

INTRO TO DATA EXPLORATION

Kaggle: from question to insight

A beginner-first tour of competitions, exploratory data analysis, and the Titanic challenge.

TODAY: EXPLORE

NEXT: PREDICT

Our mission today

1

Understand the arena

What Kaggle is, how a challenge is organized, and what you submit.

2

Inspect the evidence

Read a dataset, assess its quality, and ask useful questions.

3

Tell a small story

Use summaries and charts to explain one meaningful pattern.

Our rule: understand the data before asking a model to learn from it.

What is Kaggle?

A data science platform where a **question**, a **dataset**, and a clear **evaluation** turn learning into a hands-on challenge.

Datasets

Find and share real data.

Notebooks

Run code in the browser.

Competitions

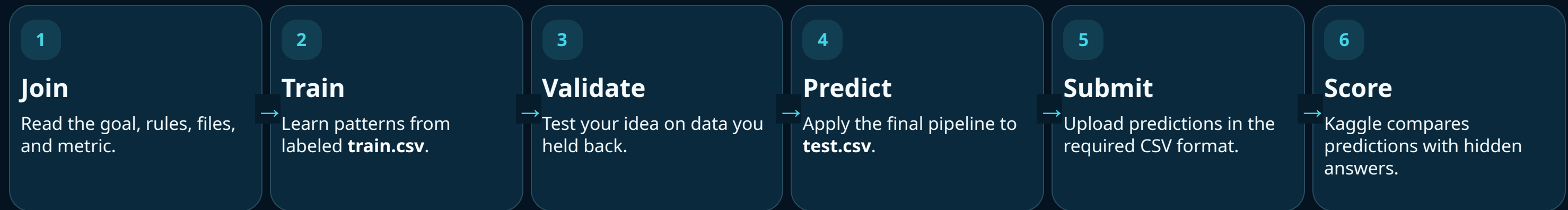
Solve defined problems.

Community

Discuss and learn together.



How a prediction challenge works



Leaderboard ≠ learning. Use it as feedback, not as the only goal. The private score helps expose overfitting to the public leaderboard.

The data analysis process

01

Frame

Define the question and success.

02 · TODAY

Load & audit

Shape, types, quality, target.

03 · TODAY

Clean

Missing values, duplicates, labels.

04 · TODAY

Explore

Distributions and relationships.

05

Engineer

Create useful, valid features.

06

Model

Fit a simple baseline first.

07

Evaluate

Measure on unseen data.

08

Communicate

Explain insight, limits, next step.

↻ New evidence sends us back to better questions.

Your role: data detective

45

minutes · pairs or trios

- 1 **Open the Titanic data** and identify the target.
- 2 **Audit its quality:** size, types, missingness, duplicates.
- 3 **Ask three questions** about who survived.
- 4 **Create three charts** that answer those questions.
- 5 **Share one insight** and one limitation with the class.

Deliverable: one notebook with readable code, three labeled charts, and a three-sentence conclusion.

The essential toolkit

Pandas

pd

Tables, cleaning, filtering, grouping, and summaries.

NumPy

np

Fast arrays, numerical operations, and missing-value logic.

Seaborn

sns

Statistical charts with concise, readable commands.

Matplotlib

plt

Precise control of figures, labels, axes, and layout.

scikit-learn

sk

Preprocessing, train/validation splits, models, and metrics.

Think of them as a team: **Pandas prepares, Seaborn reveals, scikit-learn learns.**

Start with questions — not charts

A

What does one row represent?

A passenger, transaction, match, image, or moment?

B

What are we trying to explain?

Find the target and understand its possible values.

C

Can we trust the data?

Check missing, impossible, duplicated, or inconsistent values.

D

Which groups look different?

Compare distributions — but do not claim causation.

Your dataset health report

Shape

How many rows and columns?

Data types

Numeric, categorical, text, date?

Missingness

Count and % missing per column.

Cardinality

How many unique values?

Distribution

Min, max, mean, median, quartiles.

Target balance

How common is each outcome?

Also inspect: duplicated rows, impossible values, outliers, and potential data leakage.

Titanic: Machine Learning from Disaster

The question: can passenger information help us predict who survived?

Target: **Survived** — 1 = yes, 0 = no.

891

training rows

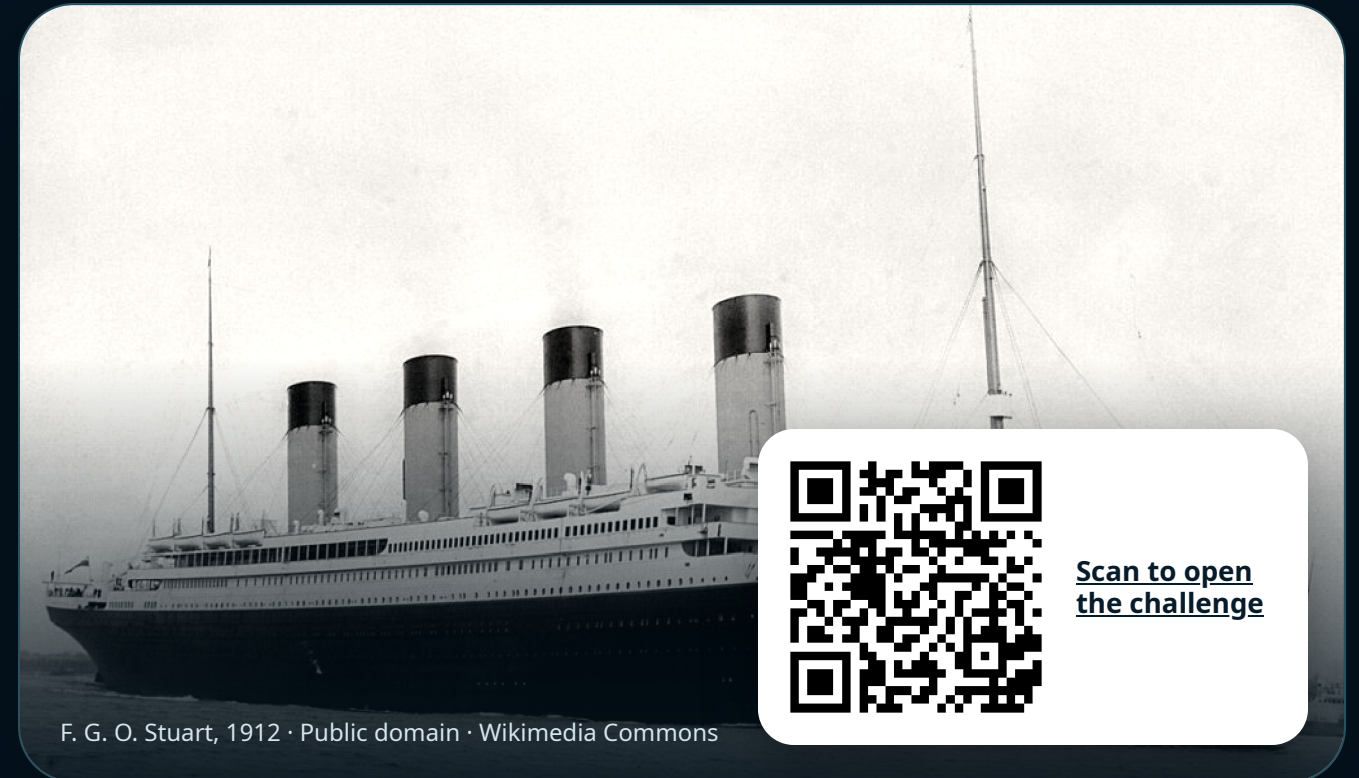
418

test rows

~38%

survived in train

Goal: learn from labeled passengers, then predict 0 or 1 for every passenger in the test set.



Scan to open
the challenge

Meet the columns

IDENTITY & CONTEXT

`PassengerId` · identifier

`Name` · name and title

`Sex` · recorded sex

`Age` · age in years

JOURNEY

`Pclass` · ticket class

`Fare` · ticket fare

`Cabin` · cabin code

`Embarked` · port

FAMILY & TICKET

`SibSp` · siblings/spouses

`Parch` · parents/children

`Ticket` · ticket code

`Survived` · target

Three early clues: `Cabin` is mostly missing, `Age` is partly missing, and several columns are categorical text.

Your first inspection, line by line

```
import pandas as pd

train = pd.read_csv(
    "/kaggle/input/competitions/titanic/train.csv"
)

train.head()
train.shape
train.info()
train.describe(include="all")
train.isna().mean().sort_values(ascending=False)
train["Survived"].value_counts(normalize=True)
```

What each command does

<code>import pandas as pd</code>	Loads Pandas and gives it the short name pd .
<code>pd.read_csv(...)</code>	Reads the CSV file into a table called train .
<code>head()</code>	Shows the first five rows for a quick visual check.
<code>shape</code>	Returns (rows, columns) — here, 891 × 12.
<code>info()</code>	Lists column names, types, and non-missing counts.
<code>describe(...)</code>	Summarizes numeric and categorical columns.
<code>isna().mean()...</code>	Calculates and ranks the missing proportion per column.
<code>value_counts(...)</code>	Measures the proportion of 0 and 1 in the target.

No — this is not the only code

The first cell **checks the data**. Next, short commands answer questions with tables and charts.

1 · Compare groups

```
# Survival rate by sex
train.groupby("Sex")["Survived"].mean()

# Survival rate by ticket class
train.groupby("Pclass")["Survived"].mean()
```

2 · Turn questions into charts

```
import seaborn as sns

sns.countplot(data=train, x="Survived")
sns.barplot(data=train, x="Sex", y="Survived")
sns.histplot(data=train, x="Age", hue="Survived")
```

Today · Part 1

Load and audit the dataset.

Today · Part 2

Group, compare, and visualize.

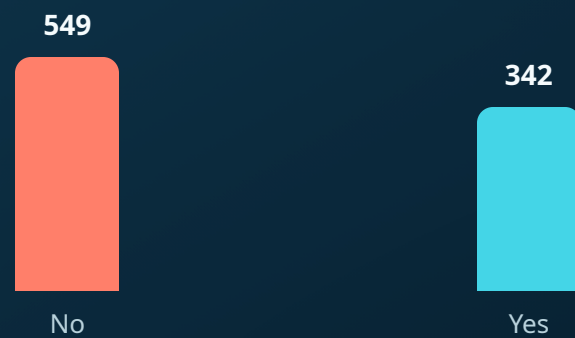
Later

Clean, model, evaluate, and submit.

Three charts, three questions

Target count

Is survival balanced?



→ What is our baseline?

Survival by sex

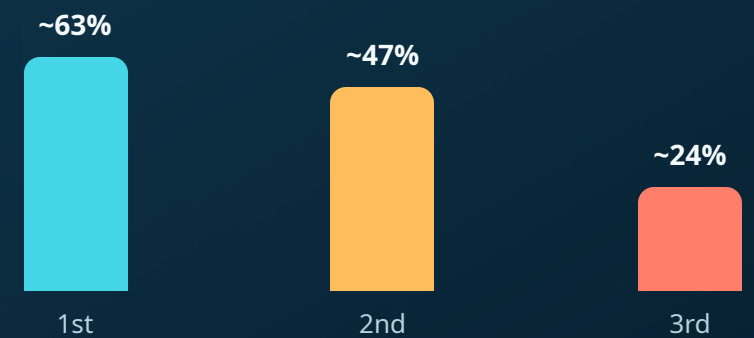
Do group rates differ?



→ Association or cause?

Survival by class

Does ticket class matter?



→ What else is related?

From exploration to a baseline

Kaggle metric: Accuracy

**correct predictions ÷
all predictions**

Example: 80 correct out of 100 → 0.80 accuracy

Accuracy is easy to read, but can be misleading when one class is very rare. Later, inspect precision, recall, F1, and the confusion matrix.

Simple rule

A transparent benchmark: always predict the majority class.

Logistic Regression

A strong, interpretable first model for binary outcomes.

Decision Tree

Learns human-readable if/then splits; can overfit.

Random Forest

Combines many trees for a more stable prediction.

Board the challenge

- 1 Open the Titanic competition**
Accept the rules and create or copy a notebook.
- 2 Produce the health report**
Shape, types, missing %, target balance, and descriptive statistics.
- 3 Answer three questions**
Use at least one univariate and two comparative charts.
- 4 Write your evidence statement**
“We observed... This may suggest... We cannot conclude...”
- 5 Share one next step**
A cleaning decision or feature worth testing later.



Scan to start

kaggle.com/competitions/titanic

ONE IDEA TO REMEMBER

Good models begin with good questions.

Explore carefully. Explain honestly. Improve one step at a time.

QUESTIONS?